

BUỔI 4. DOCKER COMPOSE, DOCKER HUB VÀ MERN MULTI-CONTAINER

Thời lượng: 4 giờ

Môi trường: Máy tính cá nhân, Docker Desktop, VS Code, GitHub Codespaces

Ứng dụng: Tiếp tục ứng dụng Quản lý sinh viên của Buổi 3

Database: MongoDB Atlas

Công nghệ: Docker, Docker Compose, Docker Hub, Git/GitHub

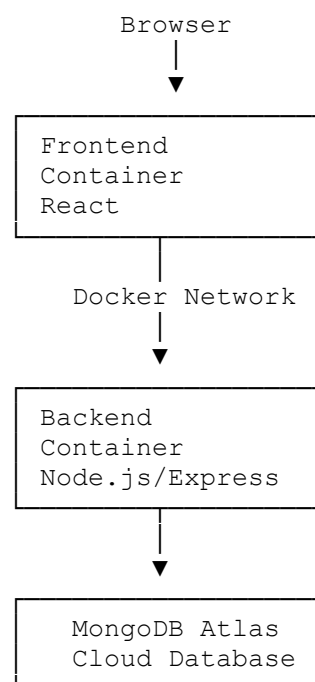
Mục tiêu

Sau khi hoàn thành buổi thực hành, sinh viên có khả năng:

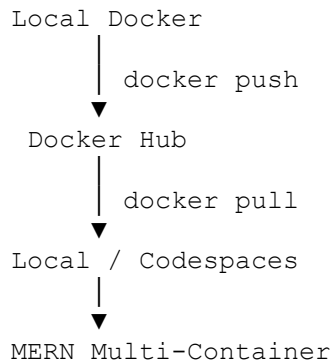
1. Tiếp tục sử dụng ứng dụng MERN đã container hóa ở Buổi 3.
2. Sử dụng Docker Compose để quản lý nhiều Container.
3. Xây dựng hệ thống MERN theo mô hình Multi-Container.
4. Hiểu mối quan hệ giữa Frontend Container, Backend Container và MongoDB Atlas.
5. Cấu hình Network và Environment Variables cho các Service.
6. Xây dựng Docker Image cho Frontend và Backend bằng Docker Compose.
7. Đẩy Docker Image lên Docker Hub.
8. Tải Docker Image từ Docker Hub về máy local.
9. Sử dụng Docker Image trên Docker Hub để triển khai ứng dụng MERN.
10. Sử dụng GitHub Codespaces để chạy và kiểm tra hệ thống Multi-Container trên môi trường Cloud.
11. Quản lý source code và cấu hình Docker Compose bằng Git/GitHub.

Kết quả cần đạt

Cuối buổi, sinh viên xây dựng được hệ thống MERN Multi-Container theo mô hình:



Đồng thời, Docker Image được quản lý trên:



PHẦN 1. TIẾP TỤC SOURCE CODE BUỔI 3

Thời gian dự kiến: 20 phút

Câu 1. Mở Repository `cloud-lab` trên GitHub và kiểm tra source code đã được Push ở Buổi 3. (Gợi ý: Truy cập Repository trên GitHub.)

Câu 2. Mở Terminal hoặc Git Bash trên máy tính cá nhân. (Gợi ý: Sử dụng Git Bash hoặc Terminal trong VS Code.)

Câu 3. Di chuyển vào thư mục `cloud-lab` đã sử dụng ở Buổi 3. (Gợi ý lệnh: `cd cloud-lab`)

Câu 4. Kiểm tra Remote Repository. (Gợi ý lệnh: `git remote -v`)

Câu 5. Cập nhật source code mới nhất từ GitHub. (Gợi ý lệnh: `git pull`)

Câu 6. Kiểm tra trạng thái source code sau khi cập nhật. (Gợi ý lệnh: `git status`)

Câu 7. Kiểm tra cấu trúc ứng dụng MERN và các file Docker đã xây dựng ở Buổi 3. (Gợi ý: Kiểm tra thư mục `client`, `server`, `Dockerfile` và `docker-compose.yml`.)

Câu 8. Kiểm tra phiên bản Docker và Docker Compose. (Gợi ý lệnh: `docker --version` và `docker compose version`)

Mục tiêu: Sinh viên tiếp tục sử dụng chính ứng dụng MERN đã hoàn thành ở Buổi 3 làm nền tảng cho bài thực hành Multi-Container.

PHẦN 2. DOCKER COMPOSE VÀ MERN MULTI-CONTAINER

Thời gian dự kiến: 50 phút

- Câu 9.** Kiểm tra nội dung file `docker-compose.yml` của ứng dụng MERN. (Gợi ý: Kiểm tra các Service `frontend` và `backend`.)
- Câu 10.** Kiểm tra cấu hình Build của Service `backend`. (Gợi ý: Kiểm tra thuộc tính `build`.)
- Câu 11.** Kiểm tra cấu hình Build của Service `frontend`. (Gợi ý: Kiểm tra thuộc tính `build`.)
- Câu 12.** Kiểm tra cấu hình Port của các Service. (Gợi ý: Kiểm tra thuộc tính `ports`.)
- Câu 13.** Kiểm tra cấu hình biến môi trường của Backend. (Gợi ý: Kiểm tra `MONGODB_URI` và `PORT`.)
- Câu 14.** Xây dựng lại Docker Image cho toàn bộ ứng dụng MERN. (Gợi ý lệnh: `docker compose build`)
- Câu 15.** Khởi động toàn bộ các Service ở chế độ Background. (Gợi ý lệnh: `docker compose up -d`)
- Câu 16.** Kiểm tra trạng thái các Service. (Gợi ý lệnh: `docker compose ps`)
- Câu 17.** Liệt kê các Container được Docker Compose tạo ra. (Gợi ý lệnh: `docker ps`)
- Câu 18.** Kiểm tra Docker Network được Docker Compose tạo ra. (Gợi ý lệnh: `docker network ls`)
- Câu 19.** Kiểm tra thông tin các Container thuộc Docker Network. (Gợi ý lệnh: `docker network inspect <network-name>`)
- Câu 20.** Xem log của toàn bộ hệ thống MERN. (Gợi ý lệnh: `docker compose logs`)
- Câu 21.** Xem log riêng của Backend Service. (Gợi ý lệnh: `docker compose logs backend`)
- Câu 22.** Truy cập Frontend bằng trình duyệt và kiểm tra ứng dụng. (Gợi ý: Truy cập `http://localhost:<port>`.)
- Câu 23.** Kiểm tra chức năng lấy danh sách sinh viên từ MongoDB Atlas. (Gợi ý: Kiểm tra luồng `React → Backend → MongoDB Atlas`.)
- Câu 24.** Thêm một sinh viên thông qua giao diện React. (Gợi ý: Sử dụng Form nhập dữ liệu.)
- Câu 25.** Thực hiện cập nhật và xóa sinh viên thông qua ứng dụng. (Gợi ý: Sử dụng các API `PUT` và `DELETE` đã xây dựng ở Buổi 2.)
- Câu 26.** Dừng toàn bộ hệ thống MERN bằng Docker Compose. (Gợi ý lệnh: `docker compose down`)
- Câu 27.** Khởi động lại hệ thống và kiểm tra ứng dụng. (Gợi ý lệnh: `docker compose up -d`)
- Mục tiêu:** Sinh viên sử dụng Docker Compose để quản lý ứng dụng MERN gồm nhiều Container thay vì chạy từng Container độc lập.

PHẦN 3. ĐÓNG GÓI DOCKER IMAGE VÀO DOCKER HUB

Thời gian dự kiến: 50 phút

Câu 28. Truy cập Docker Hub và đăng nhập tài khoản Docker. (Gợi ý: Truy cập Docker Hub bằng trình duyệt.)

Câu 29. Kiểm tra username Docker Hub của tài khoản. (Gợi ý: Xem thông tin tài khoản trên Docker Hub.)

Câu 30. Đăng nhập Docker Hub từ Terminal. (Gợi ý lệnh: `docker login`)

Câu 31. Liệt kê các Docker Image của ứng dụng MERN đang có trên máy local. (Gợi ý lệnh: `docker images`)

Câu 32. Xác định Docker Image Backend được tạo ở Buổi 3. (Gợi ý: Tìm Image `mern-backend` trong danh sách Image.)

Câu 33. Đổi tên Docker Image Backend theo cú pháp của Docker Hub. (Gợi ý lệnh: `docker tag mern-backend <dockerhub-username>/mern-backend:1.0`)

Câu 34. Đổi tên Docker Image Frontend theo cú pháp của Docker Hub. (Gợi ý lệnh: `docker tag mern-frontend <dockerhub-username>/mern-frontend:1.0`)

Câu 35. Kiểm tra các Tag của Docker Image Backend và Frontend. (Gợi ý lệnh: `docker images`)

Câu 36. Push Docker Image Backend lên Docker Hub. (Gợi ý lệnh: `docker push <dockerhub-username>/mern-backend:1.0`)

Câu 37. Push Docker Image Frontend lên Docker Hub. (Gợi ý lệnh: `docker push <dockerhub-username>/mern-frontend:1.0`)

Câu 38. Kiểm tra các Repository Docker Image trên Docker Hub. (Gợi ý: Kiểm tra hai Repository `mern-backend` và `mern-frontend`.)

Câu 39. Kiểm tra Tag 1.0 của Docker Image Backend trên Docker Hub. (Gợi ý: Mở Repository `mern-backend` trên Docker Hub.)

Câu 40. Kiểm tra Tag 1.0 của Docker Image Frontend trên Docker Hub. (Gợi ý: Mở Repository `mern-frontend` trên Docker Hub.)

Mục tiêu: Sinh viên hiểu được quy trình:

Source Code

↓

Docker Build

↓
Docker Image
↓
docker tag
↓
Docker Hub

PHẦN 4. PULL IMAGE TỪ DOCKER HUB

Thời gian dự kiến: 35 phút

Câu 41. Xóa Docker Image Backend trên máy local. (Gợi ý lệnh: `docker rmi <dockerhub-username>/mern-backend:1.0`)

Câu 42. Xóa Docker Image Frontend trên máy local. (Gợi ý lệnh: `docker rmi <dockerhub-username>/mern-frontend:1.0`)

Câu 43. Kiểm tra danh sách Docker Image sau khi xóa. (Gợi ý lệnh: `docker images`)

Câu 44. Tải Docker Image Backend từ Docker Hub về máy local. (Gợi ý lệnh: `docker pull <dockerhub-username>/mern-backend:1.0`)

Câu 45. Tải Docker Image Frontend từ Docker Hub về máy local. (Gợi ý lệnh: `docker pull <dockerhub-username>/mern-frontend:1.0`)

Câu 46. Kiểm tra các Docker Image vừa được tải về. (Gợi ý lệnh: `docker images`)

Câu 47. Chạy Backend Container từ Docker Image được tải từ Docker Hub. (Gợi ý: Sử dụng `docker run` và ánh xạ port 5000.)

Câu 48. Kiểm tra Backend API sau khi chạy từ Docker Image trên Docker Hub. (Gợi ý lệnh: `curl http://localhost:5000/api/students`)

Câu 49. Dừng và xóa Backend Container vừa tạo. (Gợi ý lệnh: `docker stop` và `docker rm`.)

Mục tiêu: Sinh viên hiểu được Docker Hub đóng vai trò như một Registry để lưu trữ và phân phối Docker Image.

PHẦN 5. TRIỂN KHAI MERN MULTI-CONTAINER TỪ DOCKER HUB

Thời gian dự kiến: 45 phút

Ở phần này, sinh viên không Build Image trực tiếp từ source code mà sử dụng Image đã Push lên Docker Hub.

Câu 50. Tạo file `docker-compose.hub.yml` để triển khai ứng dụng từ Docker Hub. (Gợi ý: Tạo file Docker Compose mới tại thư mục gốc.)

Câu 51. Khai báo Service `backend` sử dụng Docker Image từ Docker Hub. (Gợi ý: Sử dụng thuộc tính `image` thay cho `build`.)

Câu 52. Khai báo Service `frontend` sử dụng Docker Image từ Docker Hub. (Gợi ý: Sử dụng thuộc tính `image` thay cho `build`.)

Câu 53. Cấu hình Port cho Backend Service. (Gợi ý: Ánh xạ port `5000:5000`.)

Câu 54. Cấu hình Port cho Frontend Service. (Gợi ý: Ánh xạ port Frontend ra máy local.)

Câu 55. Cấu hình biến môi trường `MONGODB_URI` cho Backend Service. (Gợi ý: Sử dụng thuộc tính `environment`.)

Câu 56. Khởi động hệ thống MERN bằng Docker Compose sử dụng Image từ Docker Hub. (Gợi ý lệnh: `docker compose -f docker-compose.hub.yml up -d`)

Câu 57. Kiểm tra các Service đang chạy. (Gợi ý lệnh: `docker compose -f docker-compose.hub.yml ps`)

Câu 58. Kiểm tra các Docker Image được sử dụng bởi các Container. (Gợi ý lệnh: `docker ps`)

Câu 59. Truy cập Frontend bằng trình duyệt. (Gợi ý: Truy cập `http://localhost:<port>`.)

Câu 60. Hiển thị danh sách sinh viên từ MongoDB Atlas. (Gợi ý: Kiểm tra luồng `React Container → Backend Container → MongoDB Atlas`.)

Câu 61. Thêm một sinh viên thông qua Frontend Container. (Gợi ý: Sử dụng Form nhập dữ liệu.)

Câu 62. Kiểm tra dữ liệu vừa thêm trên MongoDB Atlas. (Gợi ý: Mở Collection `students`.)

Câu 63. Dừng toàn bộ hệ thống MERN. (Gợi ý lệnh: `docker compose -f docker-compose.hub.yml down`)

Câu 64. Khởi động lại hệ thống từ Docker Image trên Docker Hub. (Gợi ý lệnh: `docker compose -f docker-compose.hub.yml up -d`)

Mục tiêu: Sinh viên triển khai được ứng dụng MERN Multi-Container chỉ từ các Docker Image được lưu trên Docker Hub.

PHẦN 6. TRIỂN KHAI MERN TRÊN GITHUB CODESPACES

Thời gian dự kiến: 40 phút

Phần này giúp sinh viên thấy được Docker Image đã Push lên Docker Hub có thể được sử dụng trên một môi trường khác mà không cần Build lại từ source code.

Câu 65. Mở Repository `cloud-lab` trên GitHub và tạo GitHub Codespace từ branch `main`. (Gợi ý: Chọn **Code** → **Codespaces** → **Create codespace on main.**)

Câu 66. Mở Terminal trong GitHub Codespaces. (Gợi ý: Chọn **Terminal** → **New Terminal.**)

Câu 67. Kiểm tra phiên bản Docker trong Codespaces. (Gợi ý lệnh: `docker --version`)

Câu 68. Kiểm tra phiên bản Docker Compose trong Codespaces. (Gợi ý lệnh: `docker compose version`)

Câu 69. Đăng nhập Docker Hub từ Codespaces. (Gợi ý lệnh: `docker login`)

Câu 70. Tải Docker Image Backend từ Docker Hub. (Gợi ý lệnh: `docker pull <dockerhub-username>/mern-backend:1.0`)

Câu 71. Tải Docker Image Frontend từ Docker Hub. (Gợi ý lệnh: `docker pull <dockerhub-username>/mern-frontend:1.0`)

Câu 72. Kiểm tra các Docker Image đã được tải về Codespaces. (Gợi ý lệnh: `docker images`)

Câu 73. Sao chép hoặc tạo file Docker Compose sử dụng Image từ Docker Hub. (Gợi ý: Sử dụng `docker-compose.hub.yml`.)

Câu 74. Khởi động hệ thống MERN Multi-Container trong Codespaces. (Gợi ý lệnh: `docker compose -f docker-compose.hub.yml up -d`)

Câu 75. Kiểm tra các Container đang chạy trong Codespaces. (Gợi ý lệnh: `docker compose -f docker-compose.hub.yml ps`)

Câu 76. Forward Port của Frontend trong GitHub Codespaces. (Gợi ý: Mở mục **PORTS** và chọn Port của Frontend.)

Câu 77. Truy cập ứng dụng MERN bằng trình duyệt thông qua URL được Codespaces cung cấp. (Gợi ý: Mở Forwarded Address của Frontend Port.)

Câu 78. Kiểm tra chức năng hiển thị danh sách sinh viên. (Gợi ý: **React Container** → **Backend Container** → **MongoDB Atlas.**)

Câu 79. Thêm, sửa và xóa sinh viên từ ứng dụng chạy trên Codespaces. (Gợi ý: Sử dụng các chức năng **CRUD** đã xây dựng ở Buổi 2.)

Mục tiêu: Sinh viên hiểu được khả năng phân phối Docker Image và triển khai cùng một ứng dụng trên nhiều môi trường khác nhau.

PHẦN 7. CẬP NHẬT IMAGE VÀ QUẢN LÝ VERSION

Thời gian dự kiến: 20 phút

Câu 80. Thay đổi một nội dung hiển thị trên React Frontend. (Gợi ý: *Chỉnh sửa giao diện hoặc tiêu đề ứng dụng.*)

Câu 81. Build lại Docker Image Frontend sau khi thay đổi source code. (Gợi ý lệnh: `docker build -t mern-frontend:2.0 ./client`)

Câu 82. Gắn Tag Docker Hub cho Docker Image Frontend phiên bản 2.0. (Gợi ý lệnh: `docker tag mern-frontend:2.0 <dockerhub-username>/mern-frontend:2.0`)

Câu 83. Push Docker Image Frontend phiên bản 2.0 lên Docker Hub. (Gợi ý lệnh: `docker push <dockerhub-username>/mern-frontend:2.0`)

Câu 84. Kiểm tra các Version của Frontend Image trên Docker Hub. (Gợi ý: *Kiểm tra các Tag 1.0 và 2.0.*)

Câu 85. Thay đổi Docker Compose để sử dụng Frontend Image phiên bản 2.0. (Gợi ý: *Cập nhật thuộc tính image.*)

Câu 86. Khởi động lại hệ thống bằng Frontend Image phiên bản 2.0. (Gợi ý lệnh: `docker compose up -d`)

Mục tiêu: Sinh viên hiểu được việc quản lý Version của Docker Image và triển khai phiên bản mới mà không cần chia sẻ toàn bộ source code.

PHẦN 8. GIT VÀ BÀI TẬP TỔNG HỢP

Câu 87. Kiểm tra các file đã thay đổi trong Repository. (Gợi ý lệnh: `git status`)

Câu 88. Kiểm tra file `.env` không được đưa vào Staging Area. (Gợi ý lệnh: `git status`)

Câu 89. Kiểm tra file `docker-compose.yml` và `docker-compose.hub.yml`. (Gợi ý: *Đảm bảo cấu hình Service và Image chính xác.*)

Câu 90. Đưa các file cấu hình cần thiết vào Staging Area. (Gợi ý lệnh: `git add .`)

Câu 91. Tạo Commit cho kết quả thực hành Buổi 4. (Gợi ý lệnh: `git commit -m "Complete Lab 04 Docker Hub Multi-Container"`)

Câu 92. Push source code lên GitHub. (Gợi ý lệnh: `git push`)

Bài tập tổng hợp

Câu 93. Xóa các Container MERN đang chạy và xóa Docker Image local của Frontend và Backend. (Gợi ý: Sử dụng `docker compose down` và `docker rmi`.)

Câu 94. Tải lại Frontend Image và Backend Image từ Docker Hub. (Gợi ý: Sử dụng `docker pull`.)

Câu 95. Sử dụng Docker Compose để khởi động hệ thống MERN Multi-Container từ Docker Image trên Docker Hub. (Gợi ý: Sử dụng `docker compose -f docker-compose.hub.yml up -d`.)

Câu 96. Kiểm tra Frontend, Backend và MongoDB Atlas hoạt động theo đúng mô hình. (Gợi ý: Kiểm tra luồng `React → Express → MongoDB Atlas`.)

Câu 97. Thực hiện đầy đủ chức năng thêm, xem, sửa và xóa sinh viên. (Gợi ý: Sử dụng giao diện `React`.)

Câu 98. Kiểm tra Docker Image và Version đang được sử dụng cho Frontend và Backend. (Gợi ý lệnh: `docker ps`.)

Câu 99. Kiểm tra Docker Hub và xác nhận các Docker Image đã được Push thành công. (Gợi ý: Kiểm tra các Repository `mern-frontend` và `mern-backend`.)